

Chapter 4

Design & implementation

4.1 Motivation

Accelerating the execution of neural network models is crucial for deploying efficient machine learning applications. While MLIR provides a flexible and modular framework for building domain-specific compilers, the sheer complexity of transformation choices makes manual optimization challenging and time-consuming. Reinforcement learning (RL) has recently emerged as a promising approach to automate this process by exploring sequences of compiler transformations.

Existing RL-based approaches implemented in MLIR, however, are limited in scope. They operate at the level of individual operations without accounting for the broader structure of full neural network programs. As a result, they fail to capture important interactions between operations that significantly affect runtime performance.

This project addresses this limitation by extending the MLIR-RL system (Bendib et al. 2024) to support the optimization of entire neural network programs through a richer set of transformations. Our contributions include the integration of the **fusion transformation**, a redesigned representation of the program state, a new action space, a dataset tailored for full-program optimization, and a mechanism to specify optimization decisions across entire neural network graphs.

Our main goals are:

- **Support full neural network optimization:** Extend the existing RL system to operate on complete programs, rather than isolated operations.
- **Supports a wider range of operations:** Allow the RL agent to optimize more linear algebra and general operations found in neural networks.
- **Introduce fusion transformation:** Enable the RL agent to apply fusion as part of its action space, allowing more complex and effective optimizations.
- **Reduce execution time:** Improve runtime performance by learning and applying efficient transformation sequences.

In the following sections, we will be modeling the various changes made to the reinforcement learning system to support optimization of full neural network code. This includes defining the new representation of states and actions, adapting the environment, and incorporating the fusion transformation to enable the agent to make effective optimization decisions across complete MLIR programs.

4.2 Dataset generation

We curated a new dataset specifically for our task. Since our objective is to optimize full neural network code, we required a dataset that represents sequences of Linalg operations similar to those found in real neural network models. To construct this dataset, we adopted two approaches. The first approach involves randomly synthesizing sequences of Linalg operations. The second approach consists of extracting widely used computation blocks from well-known neural network models such as ResNet and residual (He et al. 2016) blocks.

In order to ensure that the training remains computationally feasible, we imposed a limit on the sequence length. We set this limit to $L=5$, which provides a balance between keeping the training time reasonable while still allowing the agent to learn how to handle multiple operations simultaneously.

We will detail the generation of both datasets in the following sections.

4.2.1 Synthesized sequences

For this approach we generate a random sequence of length $L=5$. Each operation in the sequence takes as input the output of the previous operation, which ensures that the resulting sequence forms a meaningful chain of computations. The operations are generated with random shapes and are randomly selected from the following set of Linalg (linear algebra dialect of MLIR) operations: `add`, `matmul`, `relu`, `conv_2d` and `pooling` which are the same operations supported by the previous system (Bendib et al. 2024), while also adding two new operations widely used in neural networks models which are `sigmoid` and `softmax_2d` to allow the agent to familiarize itself with more activation functions which will greatly help the agent when faced with real world programs.

4.2.2 Blocks of NN models

We have manually extracted from the resnet (He et al. 2016) model its repeated block which is known as the resnet block. It consists of convolution and maxpooling operations with relu and batch normalization interleaved within. It's significantly different than the synthesized programs that is because, the data-flow is not as predictable compared to the former. This will give the model a more challenging task for when it decides to apply the fusion transformation and will be a great example for real world neural networks to optimize.

We have also included a simple residual (He et al. 2016) block which can be represented by this equation

$$F(x) + x$$

where F is any operation and we have opted to choose `matmul + relu` as our F for simplicity.

We also included classical blocks such as the linear block

$$activation(x * W + b)$$

with relu and sigmoid as activation layer. This block is very crucial to our task as it is regarded as the founding block of neural network and deep learning. we also included a simple convolution block made of a `conv_2d` operation followed by a relu activation.

It is also important to note that during dataset generation, we execute each generated code sequence to verify its correctness and to record its base execution time, which will later be used during the training process. As the process is computationally intensive and time-consuming, we chose to generate a dataset of 7887 for the agent to train on. The following table shows the distribution of each program types included in the final dataset.

program type	number of instances
single operations	298
NN blocks	188
synthesized programs	7401

Table 4.1: Table of each program type and its number of occurrences in the dataset

4.3 Actions

In a reinforcement learning system, an action defines what the agent decides to apply and by which the environment transitions from the current state S_t to a new state S_{t+1} . In our context, as defined in (Bendib et al. 2024) an action corresponds to applying a specific transformation (possibly with parameters) to an operation, moving to the next operation, or choosing to terminate the optimization schedule.

The original system (Bendib et al. 2024) supported the following code optimization transformations: **Tiling+Parallelization**(TP), **Tiling**(T), **Interchange**(I), **Vectorization** (V). To support optimization of full neural network programs, we introduce two additional actions:

- **Tiling + Fusion (TF)**: Applies loop tiling to the current operation and attempts to fuse it with its current producer, if fusion is semantically valid. In this context, the *producer* is an operation that generates data consumed by another operation, while the *consumer* is the operation that uses this data.

In the MLIR framework, applying fusion on its own is not possible: the Transform dialect requires the consumer operation to be tiled first before fusion can be applied. This constraint led us to combine tiling and fusion into a single transformation (similar to the approach done by (Bendib et al. 2024), where tiling and parallelization were also grouped).

This action allows the agent to perform cross-operation optimizations that improve performance by reducing memory traffic and increasing locality. The number of variants for this action in the action space is $M \times M \times \dots \times M$ (N times), where M is the number of tile size options per loop and N is the number of loops.

- **Next**: It is one of the two terminal operations (which includes vectorization) it marks the schedule of the current operation as finished and as a result the agent stops applying transformations on the current operation and the environment will move directly to the next one. This action will give the agent more flexibility which helps it avoid applying suboptimal schedules if the agent judges that applying any other transformation will hurt the final execution time.

Together, these actions let the agent create more flexible optimization schedules that work across multiple operations. This expanded action space helps the system make better decisions by considering both local changes and the overall structure of the program.

4.4 State and Observation

To enable the reinforcement learning agent to make informed optimization decisions, MLIR programs must be encoded into a structured format that captures the essential computational characteristics and dependencies between operations. Our approach leverages the Abstract Syntax Tree (AST) structure -inspired by (reference tiramisu)- inherent in MLIR to construct a hierarchical tree representation that preserves the program’s structural information while providing the numerical features necessary for the RL agent.

Since our input consists of a sequence of Linalg operations, directly lowering the MLIR code to the affine level is not straightforward. This is primarily due to the absence of a one-to-one correspondence between individual Linalg operations and their affine representations. To address this, we adopt a strategy where each Linalg operation is isolated and lowered independently into the affine dialect. By performing this lowering on a per-operation basis, we ensure that the resulting affine code accurately captures the semantics of each Linalg operation without introducing dependencies or ambiguities that could arise from a global lowering. Once the affine representation of the operation is obtained, we proceed to extract its feature representation, which serves as the input to our learning agent for optimization tasks.

After lowering the Linalg operation to its affine representation, we construct a tree representation of its loop structure, where each node in the tree corresponds to a computation vector for the operations at that loop level as shown in the illustration 4.1. We build such a tree for both the consumer and the producer, and together these two trees constitute the observation that will be provided to the agent.

The computation vector is a concatenation of several features extracted from the operation, namely:

- **Loop information:** By lowering the Linalg operation to the affine dialect, we are able to extract information about the loop structure—specifically, the lower bound, upper bound, and step size. In our case, the lower bound and step are always set to 0 and 1 respectively, so we discard them and retain only the upper bounds of the loops as relevant information.
- **Load access matrices:** Within the loop nests, data is loaded from one or more vectors using loop iterators. To track which iterators are used to index which vectors, we represent each load operation with an access matrix. This matrix has shape $(D, N + 1)$, where D is the number of dimensions (indices) of the loaded vector and N is the number of loop iterators. The last column accounts for constant offsets.
- **Store access matrix:** Similar to load access matrices, each store operation is represented using a $(D, N + 1)$ matrix. Since each Linalg operation performs a single store, we have one store access matrix per operation.
- **Operation counts:** We also record the number of arithmetic operations within the loop nest, including addition, subtraction, multiplication, division, logarithm, and exponential operations.

To ensure that the computation vector has a fixed size for all types of operations and loop nest depths, we specify maximum sizes for each component:

- **Loop information (N):** We fix the maximum number of nested loops per operation to 7. If an operation has fewer than 7 loops, we pad the remaining entries.
- **Load access matrices (L):** The maximum number of load operations is set to 7, based on the maximum observed in our dataset. Operations with fewer loads are padded accordingly.
- **Maximum number of indices (D):** We set the maximum number of indices in an access matrix to 7, corresponding to the highest dimension observed for vectors in our dataset. If fewer indices are present, we pad the matrix.

Under these assumptions, we can compute the shape of the computation vector as follows:

$$\underbrace{N}_{\text{loop upper bounds}} + \underbrace{L \times D \times (N + 1)}_{\text{load access matrices}} + \underbrace{D \times (N + 1)}_{\text{store access matrix}} + \underbrace{6}_{\text{operation counts}}.$$

The environment state also maintains additional information that is relevant for decision-making, such as the operation tags, the action history, the current execution time, the initial (base) execution time, among others. These state variables assist the agent in the decision-making process, for example by masking invalid actions or terminating schedules when necessary.

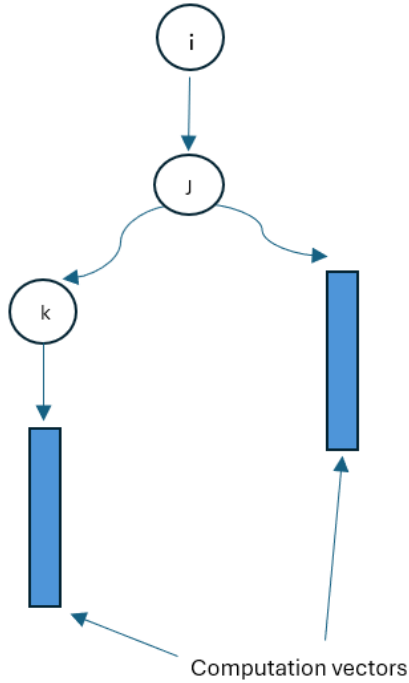


Figure 4.1: Example of an AST representation with the computation vectors

4.5 Environment

The environment is the main component of a reinforcement learning environment; it defines the setting in which the agent receives inputs, takes actions, receives rewards, and transitions to new states.

A reinforcement learning environment has two main functions: a reset function and a step function. The reset function initializes the environment to a starting state for a new episode, while the step function executes the action selected by the agent and returns the new observation, reward, and other information (such as whether the episode has ended) that the agent uses for subsequent decisions.

In the following sections we will describe each function:

4.5.1 Reset function

As mentioned previously, the reset function is responsible for initializing the environment to a starting state and returning the initial observation. In our context, this involves randomly selecting an MLIR code sample

to optimize and resetting all relevant information to its initial state.

The reset function sets the first operation to be optimized as the last Linalg operation in the code. This means that the agent begins optimization from the last consumer in the computation graph and proceeds backwards toward its producers, effectively traversing the operations in reverse order.

This design choice is motivated by the fact that many compiler optimizations, particularly those related to fusion, tiling, and scheduling, depend not only on the properties of an operation itself but also on how its results are used. Starting from the consumers allows the agent to take into account the actual usage context of each operation before deciding how to optimize its dependencies. (this needs to be double checked maybe add a source)

Each Linalg operation in the MLIR code is assigned a unique identifier. Using MLIR analysis tools, we extract all producer-consumer relationships, which are represented in the form: `producer_tag -> consumer_tag`. This allows us to efficiently determine the set of producers corresponding to the operation currently being optimized.

Algorithm 5 describes the procedure of the reset function.

Algorithm 5 Environment Reset

```

function reset(idx)
Input: Optional index idx
Output: Initial state, observation
if idx is provided then
  | Set bench_index to idx
else
  | Select a random benchmark index and assign it to bench_index
Load bench_name and benchmark_data from benchmarks_data[bench_index]
Set operation_index to last operation in benchmark
if operation has producers then
  | Set producer_tag to first producer
else
  | Set producer_tag to None
// initialize State
state = new State()
state.benchmark_data = benchmark_data
state.operation_tag = operation_tag
state.producer_tag = producer_tag
state.action_mask = initialize_action_mask()
state.action_history = []
obs = build_observation(state)
return state, obs

```

4.5.2 Step Function

The step function is the most critical component of our environment. It takes the action returned by the agent, applies it to the current state, and returns both a reward—which defines the agent’s learning policy—and the next state to be used by the environment. This function is also responsible for applying all code transformations and updating the information stored in the previous state. We have dedicated significant

effort to curating this function, testing various configurations, and evaluating which settings yield the most optimal results.

The function begins by applying the code transformation selected by the agent for the current operation. If the chosen action is fusion, the fusion transformation is invoked using information about the consumer and its producer. If the fusion is successful—meaning the transformation was semantically valid and applied correctly in the code, resulting in a new code—we update the producer information—and the current consumer operation has multiple producers, we move to the next one. If it has only one producer, or if all producers have already been fused, we mask the fusion action in subsequent steps to prevent the agent from selecting it again.

We have also observed that combining fusion and vectorization generally yields better results. Therefore, when the agent selects fusion, we mask all other transformations and allow only vectorization to be selected in the next step. It is important to note that vectorization is treated as a terminal action; once the agent selects vectorization, the schedule is terminated, as no further transformations can be applied.

Once the agent has completed the schedule for the current operation, it selects the *Next* action and moves to the next operation to be optimized by traversing backwards through the computation graph. This process is repeated until the agent has traversed the entire computation graph.

Algorithm 6 describes the procedure of the step function.

Algorithm 6 Step function of the environment

function **Step**(*state*, *action*)

Input : Current state, selected action

Output: Next state, reward

Apply the selected transformation to the current operation

if *action* is *Fusion* **then**

if *fusion successful* **then**

if *multiple producers remain* **then**

 Move to next producer

else

 Mask fusion action for next steps

else if *action* is *Vectorization* or *actions* is *No Transformation* **then**

 Mark schedule as terminated (terminal action)

Compute reward based on the execution time of applied transformations

Update transformation history and state information

if *agent selects Next action* or *schedule is terminated* **then**

 Move to the next producer operation in computation graph

if *all operations traversed* **then**

 Mark episode as finished

return updated state, reward

4.6 Policy

In reinforcement learning, the policy plays a fundamental role in defining how the agent interacts with its environment. Essentially, it determines which action the agent should take in any given state. Formally, a policy is a function π that maps each state $s \in S$ to a probability distribution over the set of possible actions. In other words, $\pi(a|s)$ gives the probability that the agent selects action a when it finds itself in state s , i.e., $\pi(a|s) = P(A_t = a | S_t = s)$.

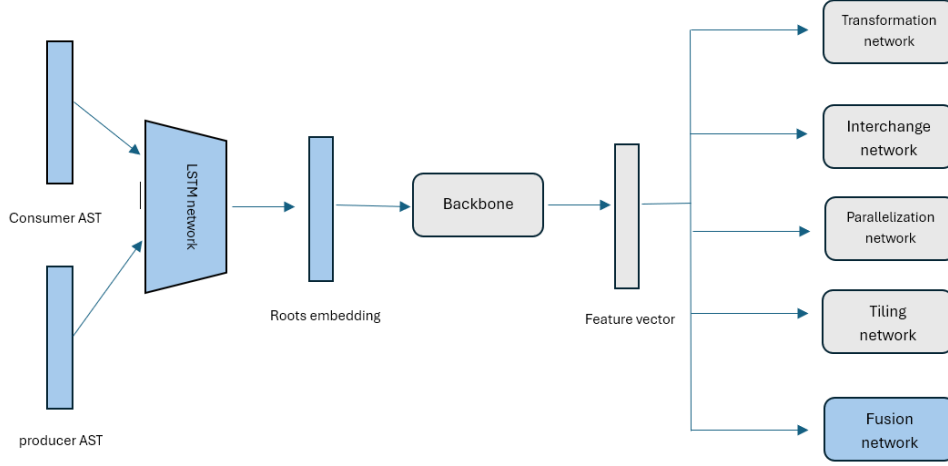


Figure 4.2: Updated policy network with the recursive loop embeddings and the fusion head

This policy acts as the agent’s strategy, guiding its behavior throughout the learning process with the goal of maximizing the total reward accumulated over time. The ultimate aim of reinforcement learning is to discover an optimal policy π^* , which leads to the highest expected return starting from any state. Associated with any policy π is its value function $V^\pi(s)$, which estimates the expected cumulative reward when the agent starts from state s and continues to act according to π .

In deep reinforcement learning, the policy is typically modeled using a trainable neural network, which learns to approximate the optimal policy by adjusting its parameters based on interactions with the environment. In our context, we updated the policy network by adding a recursive loop embedding network that takes as input the AST structure of both the producer and consumer operations and generates a new embedding that becomes the input to our backbone, which after processing passes it into the policy network and value network as shown in figure 4.2.

- **Recursive loop embedding:** This is the part responsible of encoding the AST representation of our Linalg operations into a single vector that can be used later by the rest of the network, it recursively traverses the AST structure starting from the leaves. It first starts by encoding the computation vectors at the most nested levels using the loop embedding unit as described in figure 4.3 , then recursively works it’s way back to the parent node. the loop embedding unit is composed of two LSTM’s unit one that encodes loop embedding generated from the child loop levels and the other one encodes the computation vector of the current level, it then concatenates both the embeddings and we pass it into a feed forward network as illustrated in the figure 4.4.

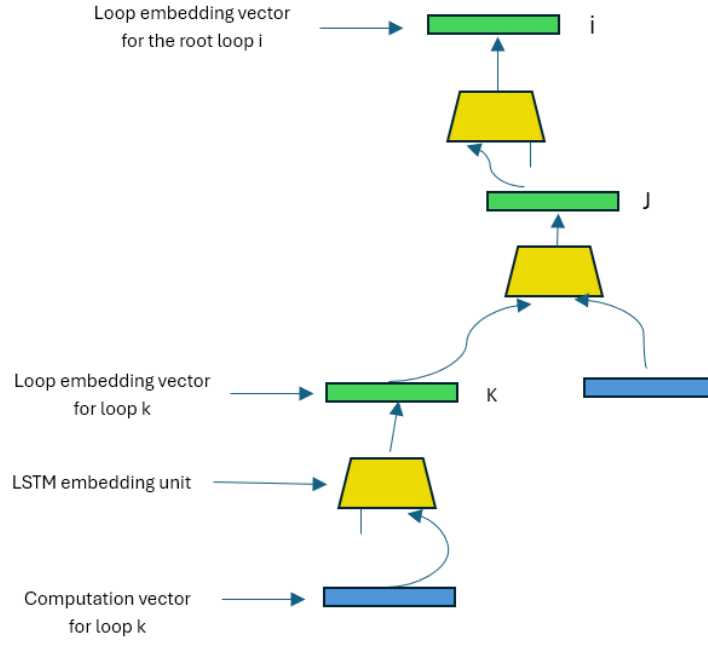


Figure 4.3: Processing of the program presented in figure 4.1 using the recursive loop embedding method

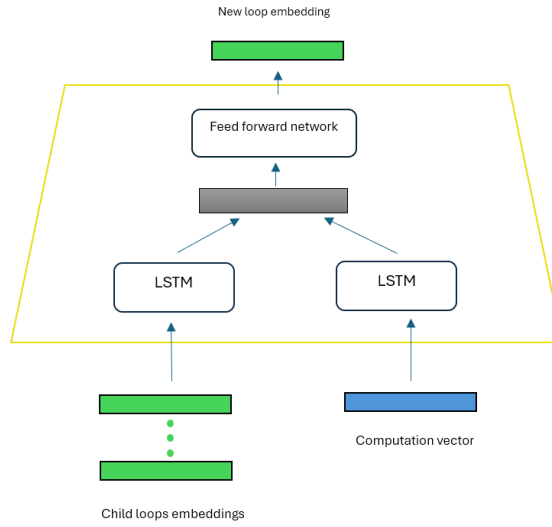


Figure 4.4: LSTM embedding unit that generates the new loop embeddings

The following algorithm 7 summarizes the procedure used to generate the loop embeddings for our AST structure.

Algorithm 7 Recursive Loop Embedding

function `get_hidden_state(node)`

Input : Root node of the Abstract Syntax Tree (AST)

Output: The final embedding for the input node

if *node has children* **then**

 Recursively compute embeddings of child nodes

 Aggregate child embeddings into a single tensor

 Pass aggregated child embeddings through the `children_nodes_LSTM` to get `child_embedding`

else

 Use default child embedding vector

if *node has computation vector* **then**

 Pass computation vector through the `computation_LSTM` to get `comp_embedding`

else

 Use default computation embedding vector

Concatenate the `child_embedding` and `comp_embedding` and Pass it through the feedforward network

return final embedding

- **Tiling + Fusion Head:** As mentioned previously, to apply fusion we must first apply tiling to the loop nest. Therefore, we need to determine the tiling parameters for the fusion transformation. Each loop has $(M+1)$ possible tiling sizes: the first one is 0, which represents no tiling, and the remaining M correspond to actual tile sizes. Similar to the Tiling + Parallelization branch, we use two fully connected layers: the first has 512 nodes, and the second has $N \times (M+1)$ nodes, where N is the number of loops, since we must determine the tile size for each loop. The output is reshaped to $(N, M+1)$, and we sample a tile size for each loop as illustrated in Figure 4.5.

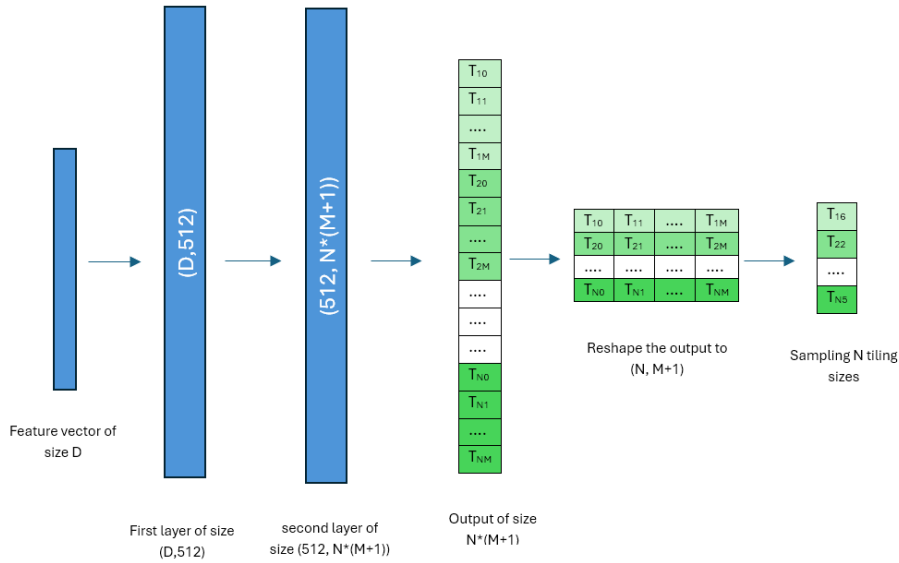


Figure 4.5: Tiling + Fusion policy network

4.7 Reward

The Agent-Environment hinges on one very crucial element, which is the reward; it conveys to the agent the performance of its actions in quantitative measures, which let him in turn adjust it for the future and eventually learn from it. It becomes clear that a well defined reward system is a defining factor for a good RL system.

Continuing on the previous system’s results (Bendib et al. 2024) where they showed that the final reward system where the agent is given the sum of the episode’s entire reward gives more performance benefits compared to rewarding the agent each time it applies an optimization (Immediate reward), when extending this system to accept full code optimization where multiple operations are to be expected, we can follow an extended final reward system where we only reward after the agent is *done* optimizing the entire code but we fear that this method delays the reward too much and does not convey the true effectiveness of the transformation to the agent.

As a result, we opt to choose a more balanced approach that incorporates the strength of **final reward** but in a more similar context as the experiments of the previous system. The agent will receive the reward only once it finishes optimizing an operation and just before transitioning to the **next** operation.

We calculate the reward with respect to the speedup achieved by the agent when optimizing the code; we calculate the execution time of the new code (after applying the optimization transformation) and compare it to the base execution time (recorded in the dataset). We reward the agent with a positive value if it achieves a speedup greater than 1; otherwise, we penalize the agent by providing a negative value if it underperforms or if the agent outputs an invalid schedule (while effectively terminating the episode).

$$r = \begin{cases} \log_{10}\left(\frac{\text{base execution time}}{\text{new execution time}}\right) & \text{if the schedule is valid} \\ -5 & \text{if applying the transformation fails} \\ -20 & \text{if an error while execution occurs} \end{cases}$$

It is important to note that -5 and -20 are values chosen by the team behind the previous system, and we did not see any benefits of altering them.

4.8 Implementation

In this section we’ll discuss the different tools and frameworks we used to implement our design. We’ll also detail how the communication between our python code and the MLIR compiler is being established and finally we’ll detail some technical parts about our training process.

4.8.1 Tools and frameworks

To support extensive experimentation and efficiently explore various configuration options in our solution, we assembled a set of tools that streamlined key aspects of our workflow. These tools aided in building the environment and neural networks, tracking experimental progress through visualizations, and comparing results across runs. The main tools used in our research are listed below.

Pytorch

PyTorch (Paszke et al. 2019) is an open-source deep learning framework developed by Facebook’s AI Research lab. It offers a flexible platform for building and training neural networks, with dynamic computation graphs and GPU acceleration capabilities that make it particularly well-suited for research. In our work, PyTorch was used to implement the models guiding the reinforcement learning agent, enabling rapid prototyping and efficient training.



Neptune

Neptune (Neptune.ai 2024) is a metadata management tool for MLOps, developed by Neptune Labs. It is designed to support the tracking and organization of machine learning experiments, promoting collaboration, reproducibility, and transparency in research workflows. Neptune enables users to log experiment metadata, monitor training metrics in real time, and compare different model versions and configurations. This functionality is especially valuable in reinforcement learning projects, where managing a large number of experiments with varying hyperparameters and settings is essential for systematic evaluation and progress tracking.



MLIR

MLIR (C. Lattner et al. 2021) is a compiler infrastructure developed by LLVM (Chris Lattner et al. 2004) with the goal of reducing the complexity involved in building machine learning frameworks and domain-specific compilers. It introduces a unified intermediate representation that enables optimization and transformation across multiple levels of abstraction. MLIR supports the creation of custom dialects and transformation passes, making it highly extensible and particularly well-suited for optimizing the kind of high-performance code generated in modern machine learning workflows.



4.8.2 MLIR Python Bindings

MLIR Python bindings (C. Lattner et al. 2021) provide Python developers with direct access to the MLIR framework from within Python. These bindings make it possible to script and automate various compilation and optimization tasks without writing C++ code. This capability is particularly valuable in our research,

as it allows seamless integration with Python-based tools and libraries, facilitating the efficient development, testing, and deployment of compiler optimizations and reinforcement learning models.

4.8.3 Computing Infrastructure

The experiments and training procedures in this work were conducted on the **Jubail High Performance Computing (HPC) cluster** at **New York University Abu Dhabi (NYUAD)**. Jubail is a medium-sized Linux-based HPC system launched in March 2022 and hosted at the NYUAD Data Center in Saadiyat. It supports large-scale computational research through powerful CPU resources by utilizing the SLURM workload manager.

While the cluster comprises **269 CPU nodes** (totaling approximately **33,984 CPU cores**), all training in this work was performed on a single CPU node.

- **Processor:** AMD EPYC 7742 (64-Core, 2.25 GHz)
- **Cores per node:** 128 (2×64 -core CPUs)
- **Memory per node:** 512 GB
- **Memory per core:** 3.75 GB

4.8.4 Training Acceleration via Caching

The training process for our reinforcement learning agent involves repeatedly applying transformation sequences, compiling the resulting MLIR code, and measuring its execution time. This compile-and-run cycle is the primary bottleneck, consuming the vast majority of the training time. Since the agent may explore the same transformation schedule for a given program multiple times, re-executing these evaluations is redundant and inefficient.

To accelerate training, we implemented a persistent caching mechanism that stores the measured execution time for any unique, transformed program. The core of this system is a key-value store implemented as a JSON file.

Key generation via hashing: A crucial design choice was how to uniquely identify a given transformed program to use as a cache key. Using the raw MLIR code as a key is impractical due to its potentially large size and variable length. Instead, we employ a stable cryptographic hash function (SHA-256) to generate a fixed-size, unique fingerprint for each code variant. This approach provides several advantages:

- **Uniformity:** All keys have a constant, short length, which is efficient for storage and lookups.
- **Uniqueness:** The probability of two different programs producing the same hash (a collision) is astronomically low, making it a reliable identifier.
- **Stability:** The hash is deterministic, ensuring that the same code will always produce the same key across different training runs, allowing the cache to be reused.

It is crucial to note that the cache is not pre-populated; rather, it is built incrementally throughout the training process. Consequently, the initial training run operates in a *cold start* mode, where every new transformation schedule results in a cache miss and incurs the full cost of compilation and execution.

However, the benefit of this approach is cumulative. Each subsequent training run leverages the cache built by all previous runs. As the agent explores the optimization space, the cache grows, and the amortized

cost of evaluating a schedule decreases significantly. This dynamic becomes particularly effective in the later stages of training, where the agent begins to converge and frequently revisits promising schedules. As a result, the cache hit rate increases, leading to a substantial acceleration of the overall training process.

4.9 Conclusion

This chapter detailed the design and implementation of our extended reinforcement learning system for optimizing full neural network programs in MLIR. We began by motivating the need to move beyond single-operation optimization to capture the complex interactions present in complete computational graphs.

The core contributions described include:

- A new dataset composed of synthesized sequences and real-world neural network blocks to train the agent on multi-operation programs.
- An expanded action space featuring **Tiling + Fusion** transformation, enabling the agent to perform inter-procedural optimizations.
- A hierarchical state representation based on the program’s AST, which provides the policy network with rich structural and computational features of both consumer and producer operations.
- A redesigned environment that traverses the program graph from consumer to producer.
- An adapted policy network architecture, featuring a recursive loop embedding model to effectively process the new AST-based state.

We also outlined the implementation details, including the software frameworks like PyTorch and MLIR, the use of caching for accelerating training runtime, and the HPC infrastructure that supported our experiments. By establishing this comprehensive system, we have laid the groundwork for training an RL agent capable of learning sophisticated, whole-program optimization strategies. The following chapter will present the experimental results and evaluate the effectiveness of this approach.

Chapter 5

Experiments and Results

This chapter details the comprehensive set of experiments conducted to evaluate our reinforcement learning agent for compiler optimization. The core challenge in this domain lies in navigating a vast and complex search space of possible transformation sequences, where optimal decisions are highly context-dependent. Traditional compilers rely on hand-tuned heuristics, which can be brittle and struggle with new or unconventional program structures. Our research is motivated by the hypothesis that a learning-based agent can discover more effective, data-driven optimization strategies. To test this, our evaluation is designed to measure the agent’s capabilities systematically, beginning with fundamental transformations and concluding with full-scale, real-world deep learning models. The objective is to present a clear and thorough picture of the agent’s current strengths, particularly its adaptability, as well as its limitations when compared to mature, production-grade systems.

We begin our analysis by focusing on **operator fusion**, a critical compiler optimization that reduces memory overhead by merging computational kernels. This section serves two purposes: first, to confirm that the agent can successfully learn to apply this powerful transformation, and second, to illustrate the inherent complexity of the task. Through targeted experiments and a manually-constructed case study, we demonstrate that the decision to fuse is not always beneficial, as it can conflict with other optimizations like parallelization. This highlights a classic compiler dilemma and underscores the necessity for an intelligent agent that can navigate such non-obvious trade-offs, providing a strong motivation for our learning-based approach.

Following this, we examine the agent’s training dynamics and the associated computational costs. An analysis of the learning curves for value loss, policy loss, entropy, and cumulative reward offers insight into the stability and convergence of the training process. We then proceed to evaluate the agent’s learned policies on individual, high-impact operations: **matrix multiplication** and **convolution**. These kernels are the computational workhorses of most neural networks, and their efficient execution is paramount. This evaluation serves as a crucial intermediate step to validate that the agent can effectively optimize these foundational building blocks before we assess its performance on more complex, interconnected graphs.

The chapter concludes with an end-to-end benchmark on three neural network models: **VGG**, **ResNet**, and **MobileNet**. We compare our agent’s results against the highly-optimized PyTorch framework. This final evaluation highlights our agent’s potential, especially on modern architectures, while also identifying its current limitations and providing clear directions for future work.

5.1 Metrics

Throughout this chapter, we aim to evaluate our method by measuring the *execution time* of programs in ms before and after letting the agent decides the schedule to apply to the programs. We also will be calculating the *speedup* generated by the agent’s schedules. Speedup are defined as follow:

$$Speedup = \frac{old_execution_time}{new_execution_time}$$

a value of speedup strictly bigger than 1 means that the schedule is good and it improved the execution time. The smaller the resulting *execution time* is the better while the bigger *speedup* is the better and means that the agent is performing better.

5.2 Training on Linear sequences

The objective of this training is to evaluate the effectiveness of the fusion transformation and determine whether the agent is capable of learning to fuse simple blocks of operations. For this reason, we trained the agent on a dataset composed exclusively of linear blocks of the form $activation(x * W + b)$.

The following table presents the hyperparameters used for the PPO agent.

Hyperparameter	Value
Learning rate	0.001
Discount factor (gamma)	0.99
Entropy coefficient	0.01
Value function coefficient	0.5
PPO clip parameter	0.2
Number of epochs for PPO	4
Mini-batch size	64
GAE lambda	0.95

Table 5.1: PPO hyperparameters

5.2.1 Results

Table 5.2 presents the performance results of applying different transformation schedules to the **add** operation. The old and new execution times are compared, and the resulting speedup is reported. As shown, the combination of tiling, fusion, and vectorization leads to a significant improvement in performance.

Schedule	Old_exec	New_exec	Speedup
Tiled & Fused & Vectorized	8.42 ms	0.8 ms	10
Tiled	8.42 ms	8.97 ms	0.9
Tiled & Parallelized & Vectorized	8.42 ms	8.98 ms	0.9

Table 5.2: Performance comparison of different schedules applied to the add operation.

5.2.2 Interpretation

The results in Table 5.2 demonstrate the impact of different transformation schedules on the execution time of the `add` operation. We observe that the schedule combining tiling, fusion, and vectorization significantly outperforms the other configurations, achieving a 10x speedup compared to the baseline. This indicates that the agent successfully learned to apply fusion in a way that groups the `add` operation with the preceding `matmul`, enabling the compiler to generate more optimized code.

In contrast, applying only tiling, or tiling with parallelization and vectorization, yields negligible improvements or even slight regressions. This suggests that fusion plays a crucial role in performance enhancement, particularly when applied in conjunction with other optimizations like vectorization.

5.3 Case Study: The Conditional Benefit of Fusion

While previous results show that fusion can be highly effective, it is not a universally optimal transformation. The best optimization strategy often depends on a complex interplay between transformations applied across an entire computation graph. To illustrate this, we manually constructed a case study analyzing a `linalg.matmul` operation followed by a `linalg.add` similar to what we had previously.

We compare two distinct, manually-defined optimization schedules. The goal is to understand the performance trade-offs a heuristic or a learning agent would face. The baseline execution time for the unoptimized sequence was approximately 931 ms.

Schedule	Transformations Applied	New Time	Speedup
Schedule A	Add: Fusion + Vectorization MatMul: Vectorization	75.8 ms	12.3x
Schedule B	Add: Tiling MatMul: Parallelization + Vectorization	53.9 ms	17.3x

Table 5.3: Performance comparison of two schedules for a `MatMul` + `Add` sequence. The baseline execution time is 931 ms.

5.3.1 Interpretation

The results in Table 5.3 highlight a classic compiler optimization dilemma.

Schedule A represents a strong, intuitive choice. Applying fusion to the `add` operation eliminates the memory-access overhead of the intermediate tensor, leading to a substantial **12.3x speedup**. This confirms that fusion is a powerful optimization for improving memory locality.

However, **Schedule B** reveals a more optimal, less obvious, path. By **not** fusing the operations, it becomes possible to apply a more aggressive parallelization and vectorization scheme to the computationally-heavy `matmul` operation. The performance gains from this enhanced parallelization outweigh the benefits of fusion, resulting in a superior **17.3x speedup**.

This case study demonstrates that the best transformation sequence is not always the one that applies the most locally-beneficial optimizations. The decision to fuse can inadvertently constrain other, more impactful

transformations like parallelization. Navigating these complex, non-linear trade-offs is extremely challenging for static heuristics. This is precisely the type of problem where a learning-based agent can provide significant value, as it can explore the vast search space and learn to identify these globally optimal, context-dependent strategies automatically.

5.4 Training on sequences

In this section we'll focus on training the model on sequences of operations as mentioned previously in our Design chapter. These sequences consist of two types: synthesized and model-extracted blocks.

The following illustrations show the training graphs for our model:

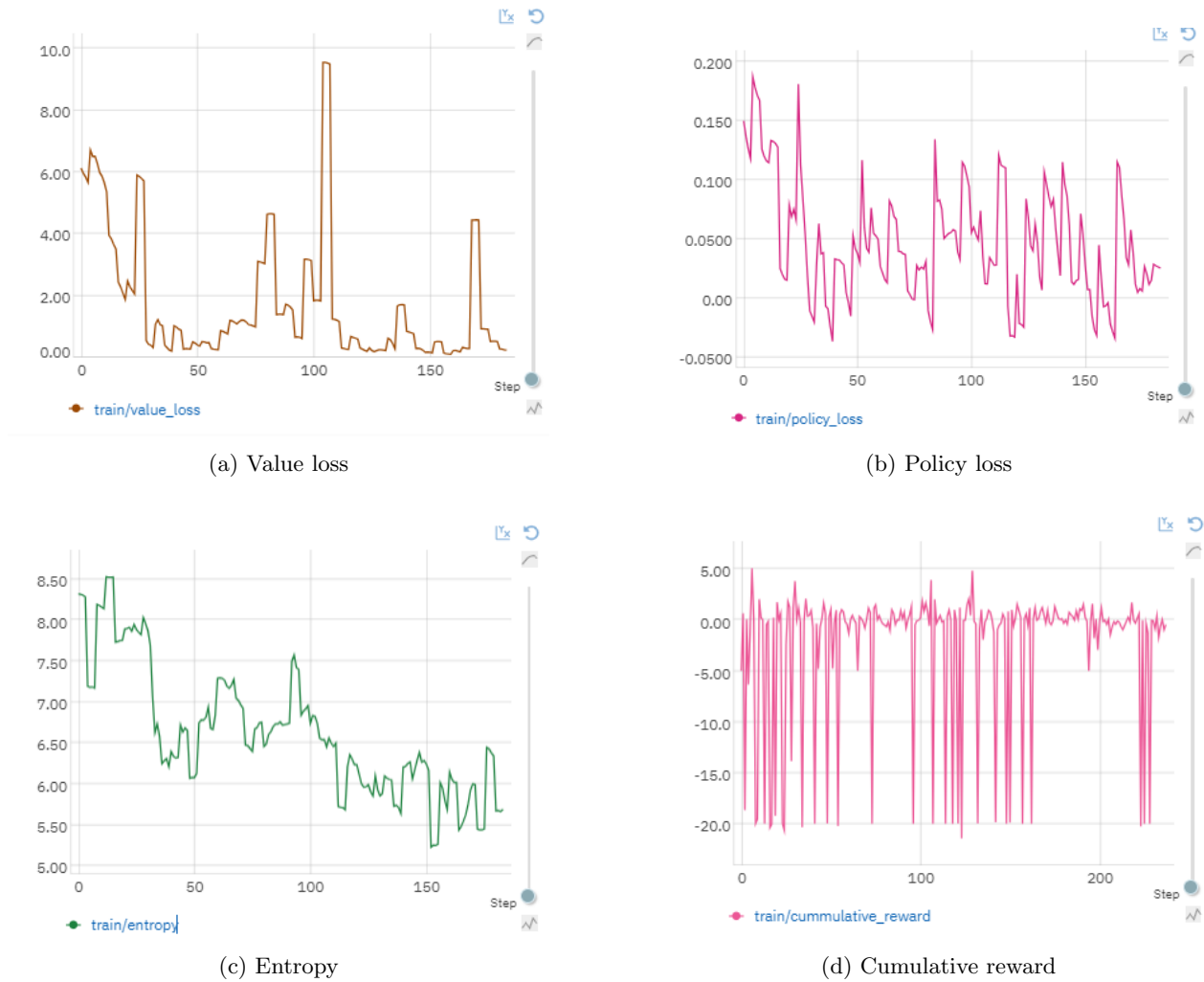


Figure 5.1: Model training results

Analysis

Figure 5.1 presents the evolution of key training metrics over time. Below is an analysis of each graph:

- **Value Loss (a):** The value loss shows significant fluctuations at the beginning of training, with several spikes, before gradually stabilizing. This indicates that the value function initially had difficulty estimating the expected returns accurately. As training progresses, the model improves its predictions, resulting in a lower and more stable loss.
- **Policy Loss (b):** The policy loss oscillates around zero without a clear upward or downward trend. This behavior is consistent with the PPO algorithm, which uses a clipped objective to prevent large policy updates. The fluctuations suggest that the policy continues to be updated moderately throughout training.
- **Entropy (c):** The entropy starts at a relatively high value and gradually decreases over time. This is expected, as high entropy during the early stages promotes exploration, while the gradual decline indicates that the agent becomes more confident and begins to exploit known strategies.
- **Cumulative Reward (d):** The cumulative reward shows high variance initially, with several negative values. Over time, it becomes more stable and increases in magnitude, suggesting that the agent is learning a more effective policy and achieving better performance.

Overall, the graphs reflect the typical learning behavior of a PPO agent. The model transitions from a phase of exploration and instability to one of more stable learning and policy improvement.

It is important to note the number of steps performed by the agent during training. The agent was trained for approximately 90 hours and completed only around 200 steps. This is a relatively small number for a reinforcement learning agent. The main reason for this low step count is that, unlike environments where a single operation is optimized, our setup involves optimizing sequences of five operations. Additionally, the environment performs an execution after each individual operation is optimized, further increasing the computational overhead and significantly slowing down the overall training process.

5.4.1 Evaluation on Single Operations

After training the agent on both blocks and sequences of operations, we proceed to evaluate its performance on individual operations before moving on to full neural networks. This intermediate evaluation step is crucial, as the ability to generate speedups on single operations serves as a strong indicator of the agent’s capacity to perform well on more complex sequences.

If the agent fails to achieve performance improvements on isolated operations, it is unlikely to produce meaningful optimizations when applied to longer and more interdependent operation sequences.

We focus our evaluation on matrix multiplication (`matmul`) and 2D convolutions (`conv`) operations, as they are among the most computationally intensive in neural networks and offer substantial potential for optimization. These operations also frequently serve as performance bottlenecks in deep learning workloads, making them ideal candidates for targeted evaluation.

We will be using for the evaluation set a manually curated set of the most used matrix multiplication and convolution operations throughout state of the art neural networks, this set was compiled by the previous system’s research team (Bendib et al. 2024).

Results

Figures 5.2 and 5.3 both show that the agent was able to find effective schedules resulting in speedups.

In Figure 5.2, we observe that most matrix multiplication (`matmul`) configurations benefited significantly from the learned transformations, with speedups reaching up to 20x in some cases. This demonstrates the agent’s ability to identify and exploit optimization opportunities in dense linear algebra operations. Notably, the agent performs particularly well on larger shapes (e.g., $256 \times 512 \times 1024$ and $256 \times 1024 \times 1024$), which are typically more computationally intensive and offer greater potential for performance gains through scheduling.

Figure 5.3 presents the results for `conv2d` operations, where the speedups are more modest but still consistent. The agent achieves speedups ranging between 2x and 10x across various convolution configurations. This indicates that while convolution operations are generally harder to optimize due to their more complex memory and compute patterns, the agent is still able to generate effective schedules. The highest gains appear for mid-sized convolutions, which likely offer a good balance between computational density and optimization flexibility.

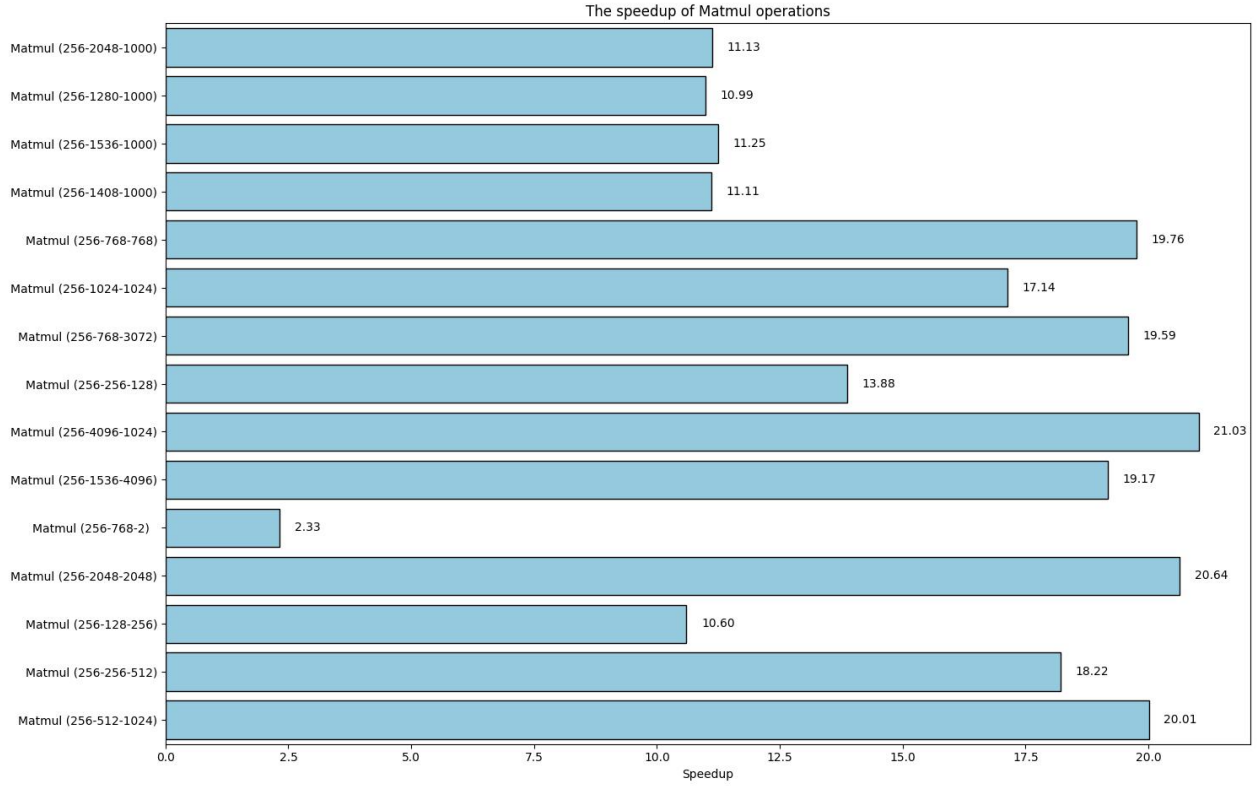


Figure 5.2: Speedups for different matmul operations

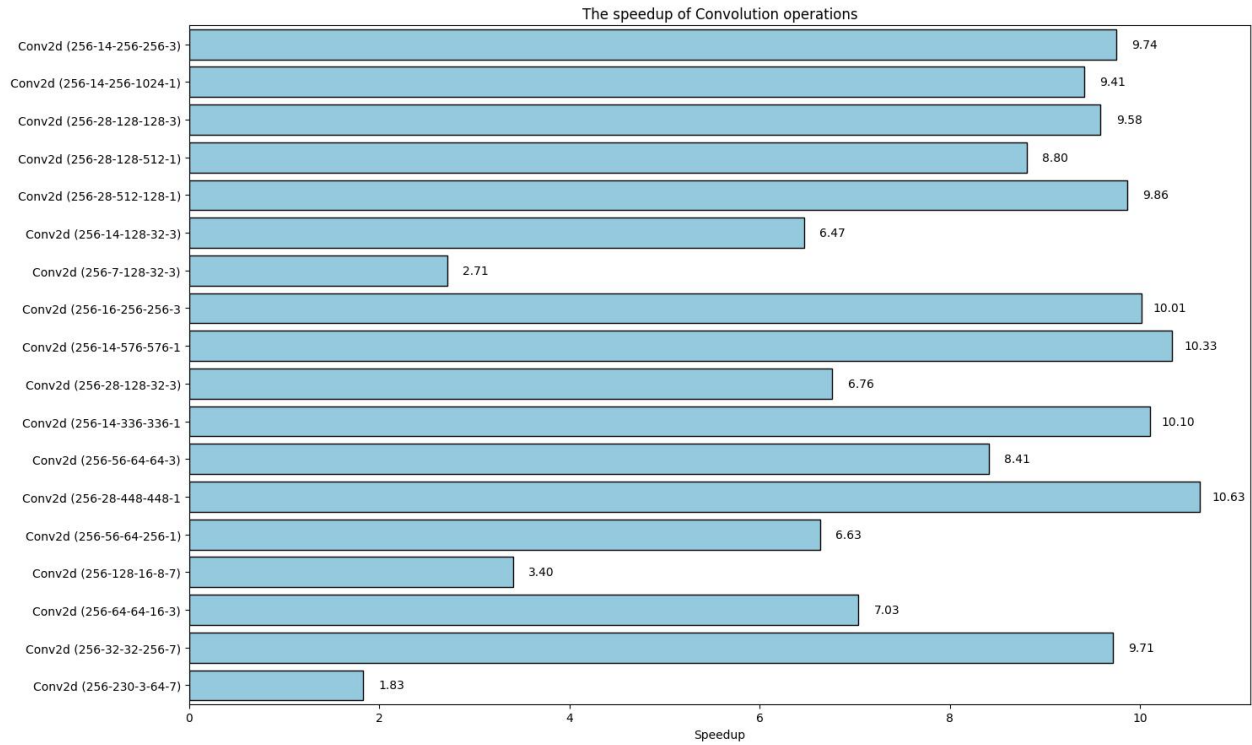


Figure 5.3: Speedups for different convolution operations

Conclusion

The evaluation on individual operations confirms the effectiveness of the agent in learning performance-enhancing schedules. Significant speedups on `matmul` and `conv2d` operations—particularly on larger and mid-sized configurations—demonstrate that the agent is capable of generalizing its learned optimization strategies beyond the training sequences. These results provide a strong foundation for extending the evaluation to full neural networks, where the ability to optimize single components is essential for achieving end-to-end performance gains.

5.4.2 Evaluation on Neural networks models

In this section, we evaluate the agent’s performance on full neural network models to assess its ability to generalize optimization strategies beyond isolated operations or short sequences. The goal is to observe whether the schedules learned during training can translate into meaningful performance improvements when applied to real-world architectures.

We selected three widely used convolutional neural network (CNN) architectures for this evaluation:

- **VGG (Visual Geometry Group):** VGG is a deep convolutional network known for its simplicity and uniform architecture. It primarily consists of stacked convolutional layers with small 3×3 kernels followed by max-pooling and fully connected layers. Despite its relatively high parameter count, VGG serves as a strong baseline for image classification tasks and is useful for evaluating performance on dense and repetitive patterns.

- **ResNet (Residual Network):** ResNet introduces residual connections that allow gradients to flow more effectively through very deep networks. This innovation enables the construction of extremely deep models without suffering from vanishing gradients. Its architecture contains residual blocks, each of which includes skip connections that bypass one or more layers. ResNet is widely used in both academic and industrial applications due to its robust performance and efficient training behavior.
- **MobileNet:** MobileNet is a lightweight neural network architecture optimized for mobile and embedded devices. It achieves efficiency by using depthwise separable convolutions, which reduce both the computational cost and the number of parameters. MobileNet is particularly suitable for performance benchmarking in low-resource environments, and it offers a useful contrast to the heavier VGG and ResNet models.

The composition of each model in terms of Linalg operation is detailed in table 5.4.

Model	Total Ops	conv_2d	pooling	matmul	fill	generic	unknown
MobileNetV2	524	35	1	1	21	448	18
ResNet	510	53	2	1	15	438	1
VGG	65	13	6	3	8	19	16

Table 5.4: Operational composition of the benchmarked neural network models.

The following table presents the speedups achieved by our reinforcement learning (RL) based optimization approach in comparison to standard PyTorch execution and PyTorch JIT.

NN Model	Our RL	PyTorch	PyTorch JIT
ResNet	2.1	4.25	4.23
MobileNet	2.5	0.41	0.39
VGG	1.09	51.81	51.52

Table 5.5: Speedups for different models using our RL-based approach, standard PyTorch execution, and PyTorch JIT (with latest optimizations).

For **ResNet**, the highest speedup is achieved by the standard PyTorch implementation (4.25x), slightly outperforming the PyTorch JIT version (4.23x). Our RL-based approach achieves a moderate speedup of 2.1x, indicating that while the agent found some effective schedules, it did not match the highly optimized static backend used by PyTorch for this architecture.

In contrast, for **MobileNet**, our RL approach significantly outperforms both PyTorch (0.41x) and PyTorch JIT (0.39x), achieving a speedup of 2.5x. This suggests that the agent is particularly effective in identifying optimizations for lightweight, mobile-friendly models that may not be well-targeted by traditional compilation heuristics.

For **VGG**, standard PyTorch and PyTorch JIT both deliver very high speedups (above 51x), which are likely due to backend-level kernel fusions and other low-level optimizations. Our RL-based approach achieves only a 1.09x speedup, indicating that the agent was not able to match the aggressive optimizations applied by the PyTorch backends in this case.

Overall, the results highlight that while our RL-based approach can yield competitive or superior performance in some cases but it may not yet match the maturity and specialization of existing compiler optimizations for highly-tuned architectures like VGG and ResNet.

Interpretation

The performance results reveal a notable difference between our agent’s optimization strategies and the highly-tuned heuristics of mature compilers. While the agent demonstrates a clear ability to find effective schedules—evidenced by the 2.5x speedup on MobileNetV2—it struggles to compete with established backends on more traditional architectures like VGG. We identify two primary factors contributing to this performance gap, both directly linked to the composition of the models detailed in Table 5.4.

First, a **limited action space and un-optimizable operations**. The set of transformations available to our agent is a subset of those in a full-fledged compiler. More importantly, our agent cannot optimize operations classified as **unknown**. This limitation is particularly impactful for VGG. As shown in Table 5.4, 16 of VGG’s 65 total operations (nearly 25%) are **unknown**. This means a significant portion of the model is fundamentally outside the scope of our agent’s optimization capabilities, severely handicapping its potential for speedup. In contrast, **unknown** operations constitute a negligible fraction of the much larger ResNet and MobileNetV2 models.

Second, a **trade-off between model scale and optimization strategy**. The scale of the model appears to dictate the most effective optimization approach.

- For **VGG**, its small size (65 operations) and regular, repetitive structure make it an ideal target for specialized, hand-crafted compiler rules. The phenomenal 51x speedup achieved by PyTorch strongly suggests the use of aggressive, hard-coded kernel fusions and low-level optimizations perfected for this common pattern. A generalist learning agent is at a disadvantage against such a specialized solution.
- For **MobileNetV2 and ResNet**, their larger scale (over 500 operations) and architectural complexity make it more difficult to apply simple, hand-tuned heuristics across the entire graph. This complexity creates a larger search space with more opportunities for optimization, which is where our RL agent excels. By exploring non-obvious combinations of transformations, the agent can discover effective schedules—like for MobileNetV2—that may be missed by static, pattern-based approaches. However, the agent’s limited training time for exploration still prevents it from matching the highly refined optimizations for ResNet.

In summary, our agent shows promise on large, complex models where manual tuning is difficult, but it is constrained by its action space and exploration budget. These limitations frame clear directions for future work: expanding the agent’s set of transformation primitives to cover more operations and developing more efficient exploration strategies to unlock complex, long-horizon optimizations.

Conclusion

Our evaluation on full neural network models reveals that our RL agent’s effectiveness is strongly dependent on model complexity and composition. The agent delivers competitive speedups on large, intricate models like MobileNetV2, where it can successfully navigate a vast search space that is challenging for static heuristics. Conversely, it struggles on smaller, regular architectures like VGG, which benefit from years of specialized, hand-tuned compiler optimizations and contain a significant portion of operations our agent cannot yet optimize. This positions our learning-based method as a powerful tool for novel or less-common architectures, while highlighting that future work must focus on expanding the agent’s action space and improving search efficiency to rival the performance of mature compilers across all domains.

General Conclusion

In this work, we extended a reinforcement learning (RL) agent to support the optimization of full neural network code. We began by updating the agent’s state and observation representations through the integration of abstract syntax tree (AST) information extracted from Linalg operations. Additionally, we expanded the action space by introducing new actions such as **next** and **fusion**, enabling the agent to effectively navigate and optimize entire neural network computation graphs.

We also constructed a new dataset composed of single operations, synthesized sequences of operations, and modules extracted from real-world models such as ResNet. This allowed us to train the agent on a diverse and representative set of inputs. To support this new dataset, we updated the environment logic to ensure compatibility and enable the agent to traverse and interact with the operations correctly.

To validate the agent’s ability to learn specific transformations, we first trained it on simple blocks of linear operations, where it successfully learned to apply the fusion transformation. We then extended training to the full dataset and evaluated the agent on complete neural network models. The evaluation showed promising results—demonstrating that the agent was able to find effective optimization schedules and, in some cases, even outperform traditional compilers like PyTorch.

In conclusion, while the results are encouraging, several challenges remain. Future work can focus on improving the agent’s performance through the following directions:

- **Parallelizing training across multiple computational nodes**, allowing the agent to explore a significantly larger portion of the search space within a reasonable time budget. This would enable the discovery of more complex and effective optimization strategies.
- **Expanding the action space** to include additional optimization techniques such as loop skewing and unrolling, thereby offering the agent a richer set of transformations to apply.
- **Replacing the LSTM-based encoder** with more powerful sequence modeling architectures, such as attention mechanisms or transformer networks, which may provide better representations of the operation sequences and improve decision-making.